COMP90050: Advanced Database Systems

THE UNIVERSITY OF MELBOURNE

Lecturer: Farhana Choudhury (PhD)

Query costs in practice

Week 2

Query costs – in practice

SQL server management studio for monitoring - Query Store

SQL Server version	Execution metric	Statistic function
SQL Server 2016 (13.x)	CPU time, Duration, Execution count, Logical reads, Logical writes, Memory consumption, Physical reads, CLR time, Degree of parallelism (DOP), and Row count	Average, Maximum, Minimum, Standard Deviation, Total
SQL Server 2017 (14.x)	CPU time, Duration, Execution count, Logical reads, Logical writes, Memory consumption, Physical reads, CLR time, Degree of parallelism, Row count, Log memory, TempDB memory, and Wait times	Average, Maximum, Minimum, Standard Deviation, Total

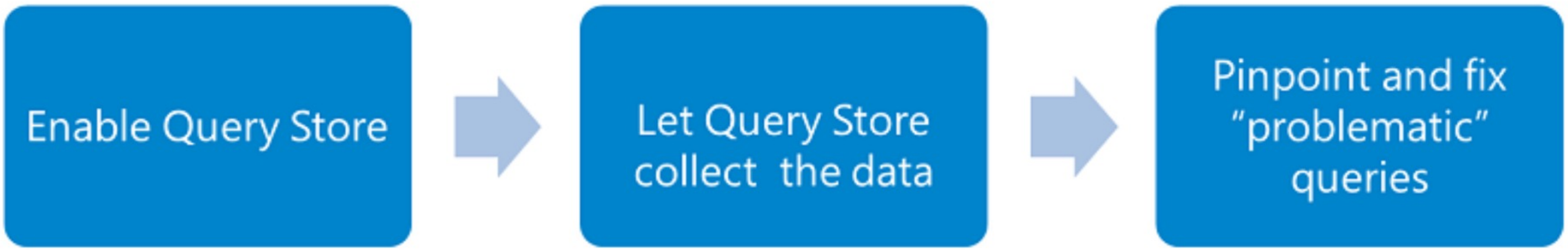
Source: https://docs.microsoft.com/en-us/sql/relational-databases/performance/best-practice-with-the-query-store?view=sql-server-ver16#Insight

Query costs – in practice

Troubleshooting to manage costs

- Identify ‘regressed queries’ - Pinpoint the queries for which execution metrics have recently regressed (for example, changed to worse).

- Track specific queries - Track the execution of the most important queries in real time.



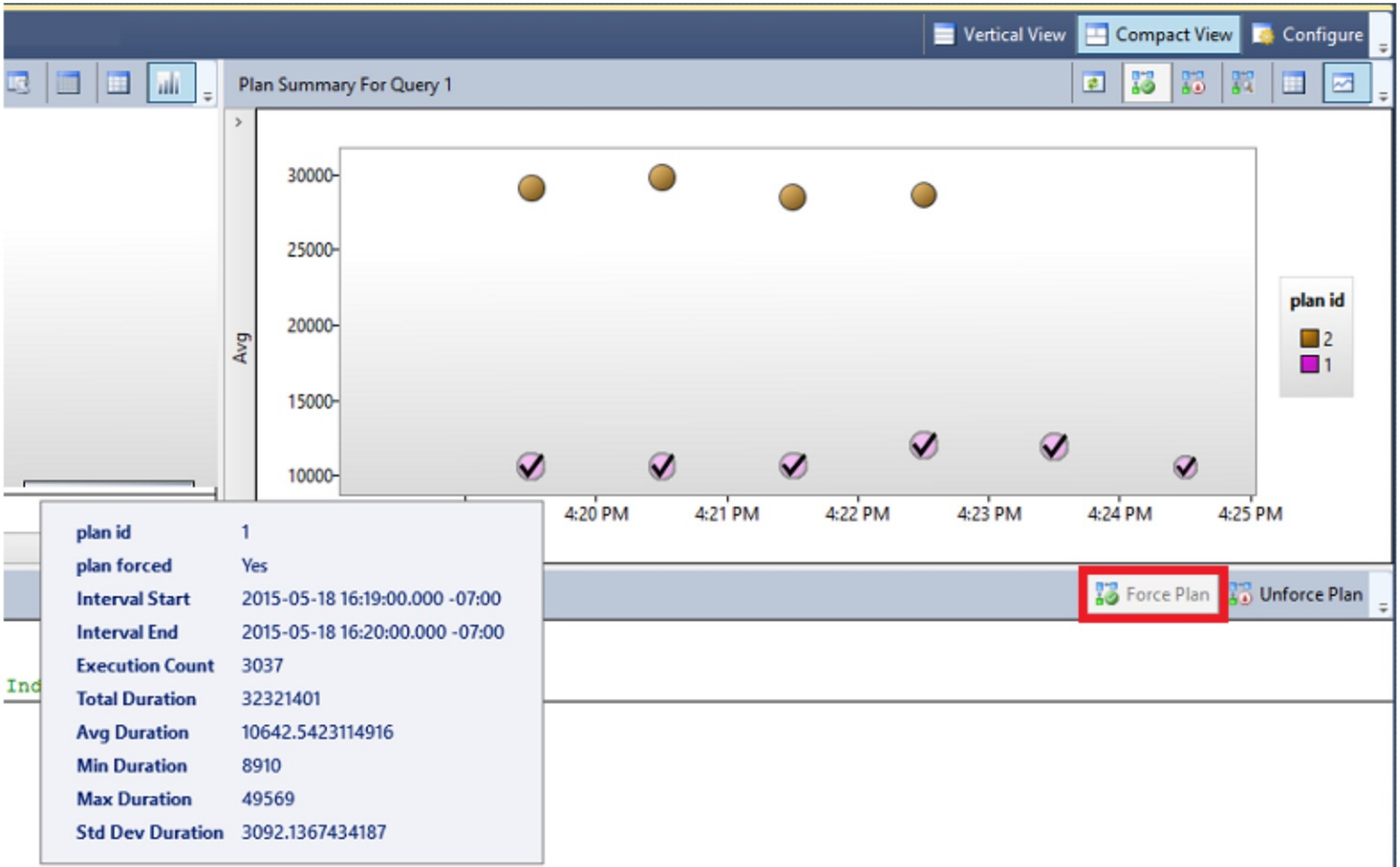
Source: https://docs.microsoft.com/en-us/sql/relational-databases/performance/best-practice-with-the-query-store?view=sql-server-ver16#Insight

Query costs – in practice

When you identify a query with suboptimal performance

- Force a query plan instead of the plan chosen by the optimizer
- Do we need an index?
- Enforce statistic recompilation
- Rewrite query?

Source: https://docs.microsoft.com/en-us/sql/relational-databases/performance/best-practice-with-the-query-store?view=sql-server-ver16#Insight



Source: https://docs.microsoft.com/en-us/sql/relational-databases/performance/best-practice-with-the-query-store?view=sql-server-ver16#Insight

Query costs – in practice

Query rewriting with parameters for execution plan reuse

SELECT * FROM Product WHERE categoryID = 1;	SELECT * FROM Product WHERE categoryID = 4;

We expect the optimizer to generate essentially the same plan and reuse the plans - parameterize

```
DECLARE @MyIntParm INT
SET @MyIntParm = 1
EXEC sp_executesql
  N'SELECT -
  FROM Product
  WHERE categoryID = @Parm',
  N'@Parm INT',
  @MyIntParm
```

Source: https://docs.microsoft.com/en-us/sql/relational-databases/query-processing-architecture-guide?view=sql-server-ver16#parameters-and-execution-plan-reuse

Can we further lower query costs?

- Store derived data
  - When you frequently need derived values
  - Data do not change frequently

- Use pre-joined tables
  - When tables need to be joined frequently
  - Regularly check and update pre-joined table for updates in the original table
  - May still return some ‘outdated’ result